

Sua missão: encontrar os projetos da pesquisa

Roteiro para a pessoa A: seleção de repositórios, metadados e funil. Primeiro, uma explicação simples; depois, o passo a passo técnico.

EXPLICANDO COMO PARA UMA CRIANÇA

Imagine que a turma vai estudar várias fábricas de brinquedos. Cada fábrica tem um nome, um endereço, uma idade e pessoas que trabalham nela. Antes de estudar como elas funcionam, alguém precisa escolher quais fábricas vão participar. **Essa é a sua missão.**

Na pesquisa, as “fábricas” são repositórios

- **Repositório:** a pasta de um projeto no GitHub, com seu código e histórico.
- **Estrelas:** pessoas que marcaram o projeto como favorito. Ajudam a representar popularidade.
- **Linguagem:** a principal linguagem usada para escrever o projeto.
- **Contribuidores:** pessoas que ajudaram a construir o projeto.
- **Idade:** há quanto tempo o repositório existe.

Você prepara a lista; os colegas investigam

PESSOA	O QUE FAZ NA ANALOGIA	NO LABORATÓRIO
A / você	Escolhe as fábricas e faz suas fichas.	Busca candidatos, coleta metadados e organiza o funil.
B	Estuda os brinquedos entregues.	Coleta releases e commits; calcula lead time.
C	Estuda as máquinas e suas falhas.	Coleta workflows; calcula CFR de CI e recuperação.

O FUNIL É UMA PENEIRA

Começamos com muitos projetos. Depois, ficam apenas os que passam pelas regras. Você registra quantos entraram, quantos ficaram e por que cada um saiu. Não basta escrever “escolhemos 100”: precisamos mostrar como chegamos a eles.

Meta da Sprint 1: o grupo entregar um pipeline funcionando para 100 repositórios, com testes, CI e introdução do artigo.

Antes de programar

Etapa 1 / combine as regras e prepare uma implementação que encaixe no projeto do grupo.

1. Leia o que realmente pertence à sua parte

No README, a parte A é: **seleção de repositórios, funil e coleta de metadados (estrelas, linguagem, contribuidores e idade)**. A Issue correspondente é a **#1**. Releases e workflows ajudam a decidir quem entra na amostra, mas seus coletores são as partes B e C.

2. Combine estas decisões com os colegas

- **Janela:** início e fim dos 12 meses fixados pelo professor. Use exatamente as mesmas datas nos três módulos; não invente a janela.
- **Popularidade:** documento o critério escolhido. O README sugere a busca **stars:>1000** como exemplo; não apresenta esse número como um filtro mínimo universal obrigatório.
- **Escolhas adicionais:** se o grupo excluir forks ou projetos arquivados, registre a justificativa. São decisões de amostragem, não exigências extras do enunciado.
- **Contribuidores e idade:** defina como contar e qual data usar como referência. Guarde a origem e as limitações.

3. Reaproveite a base já disponível

O repositório usa Node.js e já tem um cliente REST próprio em **lib/github.mjs**, desenvolvido pela parte C. Ele recebe token, pasta de cache e função de log. Evite criar outro tratamento incompatível de rate limit e cache; combine seu reaproveitamento com C.

O método **get()** devolve um objeto com **data** (resposta da API) e **next** (próxima página). Busca e metadados da parte A ainda precisam ser implementados.

ESTRUTURA SUGERIDA, AINDA A CRIAR

scripts/select-repos.mjs: comando de seleção.
lib/selection.mjs: busca, filtros e metadados.
tests/selection.test.mjs: cenários relevantes com API simulada.
config.json: regras de busca e janela; sem token.
Os nomes são uma sugestão, não arquivos já prontos.

Trabalhe na Issue #1 e em uma branch própria. Guarde GITHUB_TOKEN no ambiente. Nunca inclua credenciais no código, no CSV ou em commits.

Encontrar os candidatos

Etapa 2 / faça uma busca reproduzível e trate o limite de resultados do GitHub.

4. Consulte a busca de repositórios

```
GET /search/repositories
q=stars:>1000&sort=stars&order=desc&per_page=100
```

Monte os parâmetros com **URLSearchParams**. Salve a consulta, a data da coleta e as páginas retornadas. A primeira resposta traz **total_count**, **incomplete_results** e a lista **items**. Percorra as páginas seguindo o cabeçalho Link.

5. Divida buscas grandes em partes menores

A API retorna no máximo **1.000 resultados por consulta**. Se a busca tiver mais de 1.000 resultados, divida a faixa de estrelas e consulte cada parte. Exemplo de faixas sem sobreposição:

```
stars:1001..2000
stars:2001..5000
stars:5001..10000
stars:>10000
```

Essas faixas são apenas um começo. Se uma delas continuar acima do teto, divida novamente. Se muitos projetos tiverem o mesmo número de estrelas, use outro critério de partição, como intervalos de created, e registre a estratégia. Não trate uma lista truncada como a busca inteira.

6. Confira a integridade da seleção

- **incomplete_results=true**: registre o problema e tente novamente ou refine a busca. Não declare completude sem verificar.
- **Duplicação**: deduplique pelo ID do repositório e mantenha full_name. Partições ou consultas por linguagens podem repetir projetos.
- **Ordem**: defina uma ordenação estável, por exemplo estrelas decrescentes e full_name para desempatar; registre o critério.
- **Quantidade**: busque candidatos suficientes para repor os descartados. Uma lista inicial de 100 não garante 100 após os filtros.

EM LINGUAGEM SIMPLES

O GitHub mostra só um pedaço da lista quando ela é grande demais. Em vez de perguntar “mostre todas as fábricas”, você pergunta por grupos menores. Depois junta tudo e tira os nomes repetidos.

Saída desta etapa: lista deduplicada de candidatos e registro das consultas usadas. Comece com um lote pequeno antes de expandir.

Montar a ficha de cada projeto

Etapa 3 / colete os campos, documente as unidades e produza CSVs compatíveis.

7. Consulte os metadados

```
GET /repos/{owner}/{repo}
GET /repos/{owner}/{repo}/contributors?per_page=100&anon=true
```

CAMPO SUGERIDO	ORIGEM / COMO PREENCHER
repository_id	id da API; ajuda a evitar duplicação.
full_name / default_branch	Nome owner/repo e branch principal retornados pela API. Não presuma main.
stars / language	stargazers_count e language. Preserve language vazio se a API não informar.
contributors_count	Contagem documentada de contribuidores; defina se inclui anônimos. Não some o número de commits.
created_at / age_days	Data de criação; idade em dias até uma referência definida pelo grupo.
collected_at	Data e hora UTC da coleta, para registrar a fotografia dos dados.

Para a idade, uma opção é usar o fim da janela como referência: **age_days = (fim_da_janela - created_at) / 86.400.000** em milissegundos. Documente arredondamento e referência; repositórios criados depois dela precisam de tratamento explícito. Não misture dias, meses e anos na mesma coluna.

O README sugere contar contribuidores com per_page=1 e a última página de Link. Trate lista vazia, uma única página e respostas indisponíveis. O endpoint possui limitações de identificação/cache: registre o método e não prometa uma contagem perfeita de todas as pessoas da história do projeto.

8. Exporte duas saídas com papéis claros

- **repositories.csv**: entrada mínima de B e C, contendo full_name e default_branch. Este formato já é aceito pelo app da parte C.
- **metadata.csv**: ficha completa, ligada aos resultados de B/C pelo mesmo full_name. Documente nome, tipo e unidade de cada coluna.

```
full_name,default_branch
cli/cli,trunk
pytest-dev/pytest,main
```

O exemplo mostra o formato, não a seleção oficial. Use CSV UTF-8 com cabeçalho; não inclua token. Valores ausentes ficam vazios, com a ausência registrada; erro de API não vira automaticamente zero.

Organizar o funil com B e C

Etapas 4 / transforme os filtros em uma seleção rastreável, sem contar um projeto duas vezes.

9. Deixe cada módulo produzir a evidência correta

- **A:** candidatos, metadados e coordenação da tabela de seleção.
- **B:** quantidade de releases publicadas, sem draft e sem prerelease, dentro da janela oficial.
- **C:** existência de Actions e quantidade de runs válidos, de push no default branch, dentro da mesma janela.

O filtro mínimo obrigatório é: **pelo menos 5 releases e pelo menos 50 workflow runs válidos**. Releases corretivas ou falhas de CI não são critérios adicionais de inclusão. B e C devolvem os valores; A faz a junção e registra a decisão final.

10. Defina uma sequência para os descartes

Exemplo de sequência: candidato único → com Actions → com 5 releases → com 50 runs válidos → selecionado. A ordem deve ser fixa e documentada. Se o projeto falhar em mais de uma regra, atribua o descarte à primeira etapa que ele não passou.

ETAPA	ENTRARAM	DESCARTADOS	RESTARAM
Candidatos únicos	600	0	600
Usam Actions	600	150	450
Pelo menos 5 releases	450	200	250
Pelo menos 50 runs válidos	250	80	170
Amostra S01: regra documentada	170	70 não selecionados	100

Números acima são **fictícios**. Projetos elegíveis não escolhidos por um limite de amostra são “não selecionados”, não “inaptos”. Registre quais foram escolhidos e qual regra definiu os 100.

11. Salve também a decisão por repositório

Além do resumo **selection_funnel.csv**, mantenha **selection_decisions.csv** com full_name, releases_count, valid_runs_count, decisão e motivo. Erros 403/404/5xx ficam como pendências/erros de coleta; não são prova de que o projeto descumpra um critério.

CUIDADO COM O APP DA PARTE C

Seu CSV pode ser enviado ao app para coletar workflows. A saída de C indica o filtro de runs, mas **não substitui o filtro de releases de B**. O funil de C é parcial. A conclusão da amostra depende da combinação das três partes.

Testar, integrar e entregar

Etapa 5 / prove que a seleção funciona e deixe outra pessoa conseguir repeti-la.

12. Valide primeiro com poucos candidatos

Comece com 5 a 10 projetos. Confira manualmente alguns nomes, branches, estrelas e datas. Veja se as saídas preservam os mesmos identificadores usados por B e C. Só amplie a coleta depois de corrigir os erros desse lote.

13. Escreva testes dos riscos da seleção

- Busca acima de 1.000 resultados provoca subdivisão em vez de truncamento silencioso.
- Candidato repetido em duas consultas aparece uma única vez no resultado.
- Paginação incompleta ou erro da API não produz uma amostra “completa”.
- Ausência de linguagem, contribuidores indisponíveis e idade com referência fixa recebem tratamento documentado.
- Funil preserva as contagens e distingue descartados, não selecionados e erros.

Use respostas pequenas e simuladas, sem gastar a API nos testes. Rode **npm test** e integre os novos testes ao CI existente. A exigência de 80% do README refere-se ao módulo de métricas; sua parte precisa de verificações adequadas da seleção, sem inventar outra meta obrigatória.

14. Faça os módulos funcionarem juntos

Combine um comando que execute $A \rightarrow B/C \rightarrow$ consolidação, com a mesma configuração e janela. Seu seletor deve poder ser chamado por esse pipeline. O app web é uma interface útil; o grupo ainda precisa da execução por **um único comando documentado**.

COMANDO PROPOSTO, A IMPLEMENTAR

```
node scripts/select-repos.mjs --config config.json
```

Esse comando é uma sugestão para seu módulo. Ele só funcionará depois de você criar o script e tratar o parâmetro. Documente também o comando final do grupo quando a integração estiver pronta.

15. Use a Issue #1 como evidência de trabalho

Mantenha a Issue com assignee, tarefas e estado atualizados no GitHub Projects. Faça commits que mencionem **#1**, por exemplo **feat: coleta candidatos e metadados (#1)**. Envie o código, a documentação e as evidências permitidas; cache e credenciais ficam fora do Git.

A introdução do artigo e as hipóteses das RQs são uma entrega conjunta. Sua implementação de código é necessária para a contribuição individual; escrever o artigo sozinho não substitui os commits da Issue.

Checklist de conclusão

Use esta página para conferir o que falta antes de marcar a Issue #1 como concluída.

- ☐ Critério de popularidade e escolhas de amostragem estão documentados.
- ☐ Janela oficial e formatos de entrada/saída foram combinados com B e C.
- ☐ Busca própria via REST/GraphQL funciona com paginação e partição.
- ☐ Candidatos estão deduplicados e erros/incompletude não ficam escondidos.
- ☐ Estrelas, linguagem, contribuidores e idade foram coletados e documentados.
- ☐ repositories.csv é aceito pelos coletores de B e C.
- ☐ metadata.csv usa nomes, tipos, unidades e referências de data claros.
- ☐ Funil e motivos por repositório foram gerados pelo código.
- ☐ Dados de releases e runs foram combinados para aplicar os filtros obrigatórios.
- ☐ Pipeline integrado atende o lote de 100 da Sprint 1 e é reexecutável.
- ☐ Testes relevantes passaram, inclusive no GitHub Actions.
- ☐ Issue #1 e GitHub Projects estão atualizados; commits referenciam #1.

O QUE ENTREGAR AOS COLEGAS

1. Código de busca, seleção e metadados.
2. repositories.csv + metadata.csv.
3. selection_funnel.csv + selection_decisions.csv.
4. Configuração sem segredos, comando de execução e dicionário.
5. Testes, commits ligados à Issue #1 e evidência do CI.

Se travar, confira isto primeiro

Menos de 100 aptos? Busque mais candidatos e registre os descartes.

Branch diferente de main? Use default_branch da API.

“Zero contribuidores” após erro? Corrija: indisponível é diferente de zero.

Mais de 1.000 resultados? Particione a consulta.

Token/rate limit? Reaproveite o cliente de C e preserve o cache.

Fontes e limites deste roteiro

Base principal: README do DORA, seções 3, 4, 7 e 10. Nomes de arquivos, sequência de implementação e critérios opcionais apresentados aqui são sugestões. A janela e decisões da turma devem prevalecer.

[README do laboratório](#) | [Issue #1](#) | [Contrato e execução da parte C](#)

[GitHub: busca e limite de resultados](#) | [GitHub: contribuidores e limitações](#)